



Single source of truth

T-Mobile's unified GitOps strategy for cluster management

Joel Arends
Principal Engineer
T-Mobile

Troy Heber
Senior Architect
Red Hat

Alex Principe
Senior Consultant
Red Hat



Agenda

The Environment: ACM Hub & Spoke with GitOps and ZTP

The Challenge: Managing a bare-metal multicluster fleet at scale

The Solution: Extending GitOps Automation with OpenShift Pipelines

Image Mirroring Pipeline: Disconnected deployment automation

Cluster Build Pipeline: Full deployment automation deep dive

Next Steps: Cluster as a Service via API



The Environment

The Un-carrier Engine: Scaling 5G with Autonomous GitOps

Innovation

Pipeline Values Abstraction

A single YAML file encoding 100% of the business logic for cluster deployment, effectively serving as a de facto infrastructure API.

Autonomous Lifecycle Engine

A multi-stage OpenShift pipeline that automates everything from hardware interrogation to post-deployment cluster benchmarks.

Disconnected Deployment Automation

Automated mirroring of OCP platform and operator images to local registries for secure, air-gapped environments.

Differentiation

The "T-Mobile Layer"

Custom automation that bridges Red Hat ACM GitOps with T-Mobile specific infrastructure, including SDN, PKI Certificates, and IAM.

Zero-Touch Hardware Interrogation

Utilizing Redfish API to extract MAC addresses and validate BIOS/RAID health before any configuration is committed.

Carrier-Grade Bare Metal Scale

A unified strategy for managing a production-scale multicluster fleet with automated self-healing and continuous governance.

Business Impact

Enhanced Velocity - Cluster as a Service

Moving from manual "click-ops" to fully automated bare-metal orchestration, enabling rapid cluster deployment in hours, not days.

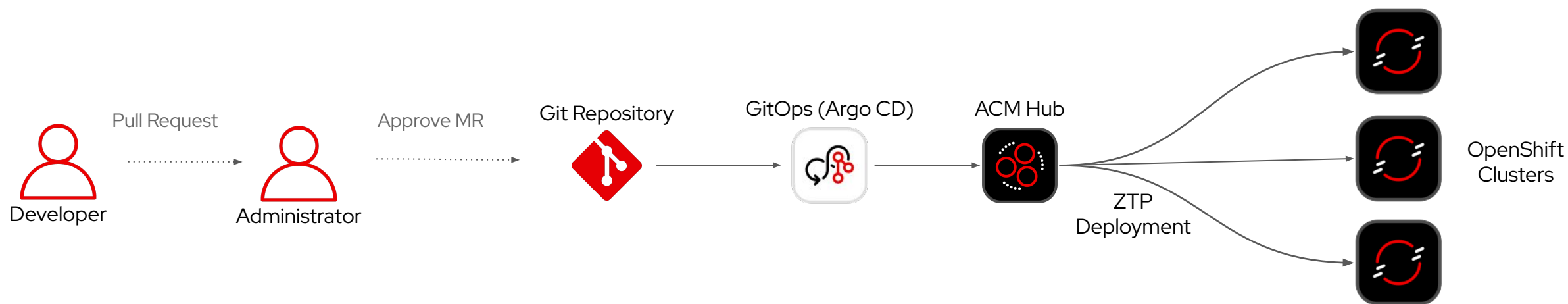
Increased Resilience

Elimination of manual configuration drift through a declarative "single source of truth" model where Git is the only interface.

Audit-Ready Efficiency

Every configuration decision is versioned and auditable in Git, redirecting engineering hours from toil back to innovation.

GitOps Delivery Model



Deploy a Red Hat ACM Hub Cluster

Day 0 – Provisioning

Bootstrap the ACM Hub Cluster

- Deploy hub on bare metal via assisted installer
- Deploy HashiCorp Vault VSO
- Deploy ACM operator
- Deploy OpenShift GitOps (Argo CD)
- Point GitOps at Git Repository

Hub becomes fully managed by GitOps

Day 1 – Bootstrap

GitOps deploys core hub services

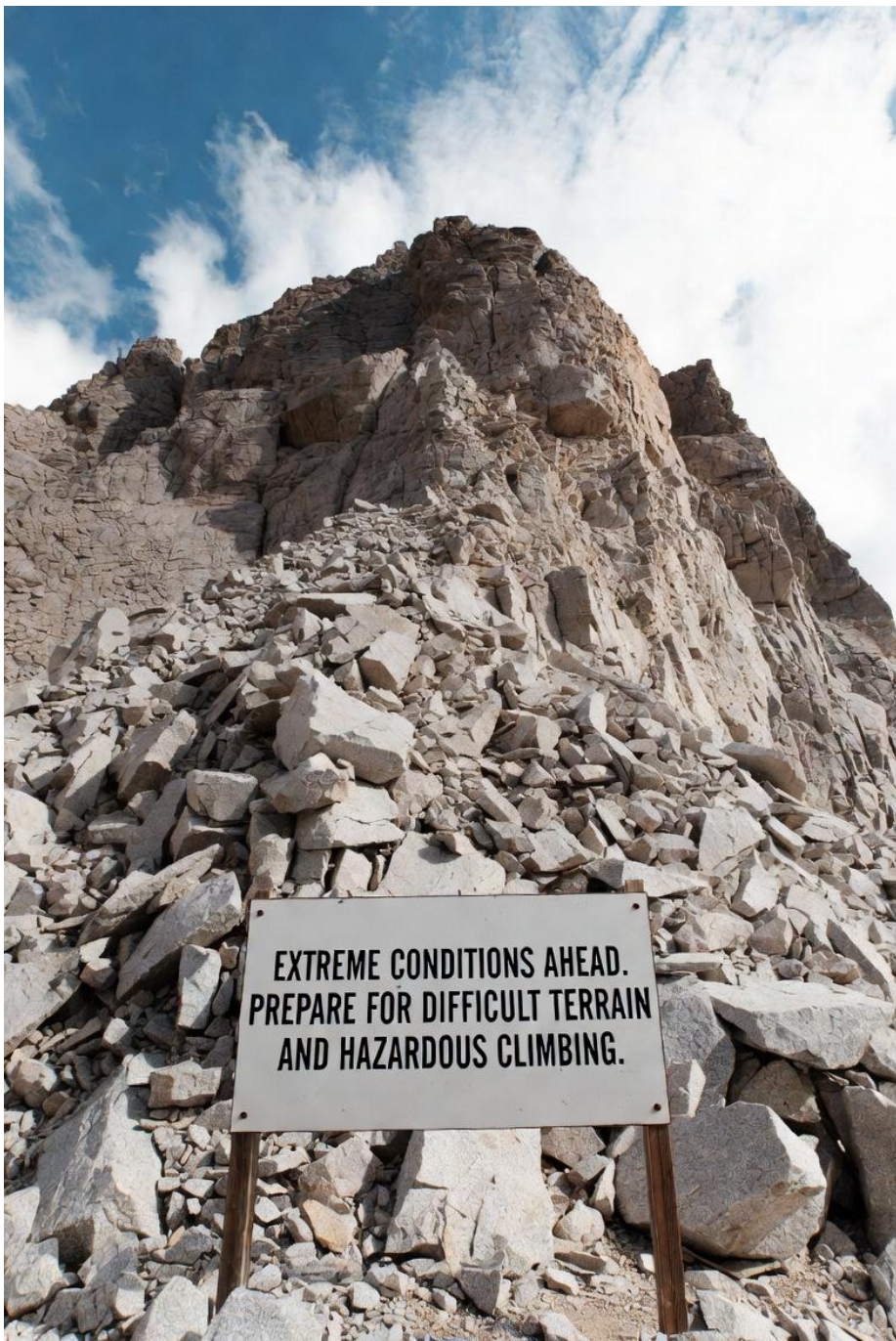
- ODF storage
- Quay Registry
- Cert-Manager PKI
- Security Operators
- CMDB Integration
- Observability

OpenShift Pipelines (Mirror, Build)

Day 2 – Management

Continuous Lifecycle Governance

- OpenShift and Operator Upgrades
- ACM Policies enforce compliance
- GitOps self-heals configuration drift



The Challenge

The Problem: Configuration Complexity at Scale

What do we need before we deploy a cluster?

Mirroring

- Have images been mirrored in Quay?

Vault

- Is it configured and accessible?
- Have the required secrets been deployed and updated?

Networking

- Has the **SDN** been configured?
- Has the **PROXY** been configured?
- Has the **FIREWALL** been configured?
- Have the **DNS** entries been added?

Nodes BMC/iDRAC/iLO

- Valid credentials and hub connectivity?
- Any node hardware problems?
- MAC addresses and network topology gathered?

Etc, etc, etc - the total list of requirements is very large!

The Problem: Configuration Complexity at Scale

ClusterInstance.yaml: **Tedious** and **Error Prone** Manual Creation

```
nodes:
  - hostName: "node1"
    role: "master"
    bmcAddress: "https://bmc-node1.example.com"
    bmcCredentialsName:
      name: "bmc-secret-node1"
    bootMACAddress: "D8:F7:1C:57:24:28"
    nodeNetwork:
      interfaces:
        - macAddress: "D8:F7:1C:57:24:28"
          name: "eno12399np0"
        - macAddress: "D8:F7:1C:57:24:2C"
          name: "ens1f0np0"
```

...

```
config:
  interfaces:
    - name: bond0
      type: bond
      ipv4:
        address:
          - ip: 10.10.10.25
            prefix-length: 24
        enabled: true
        dhcp: false
      link-aggregation:
        port:
          - eno12399np0
          - ens1f0np0
```

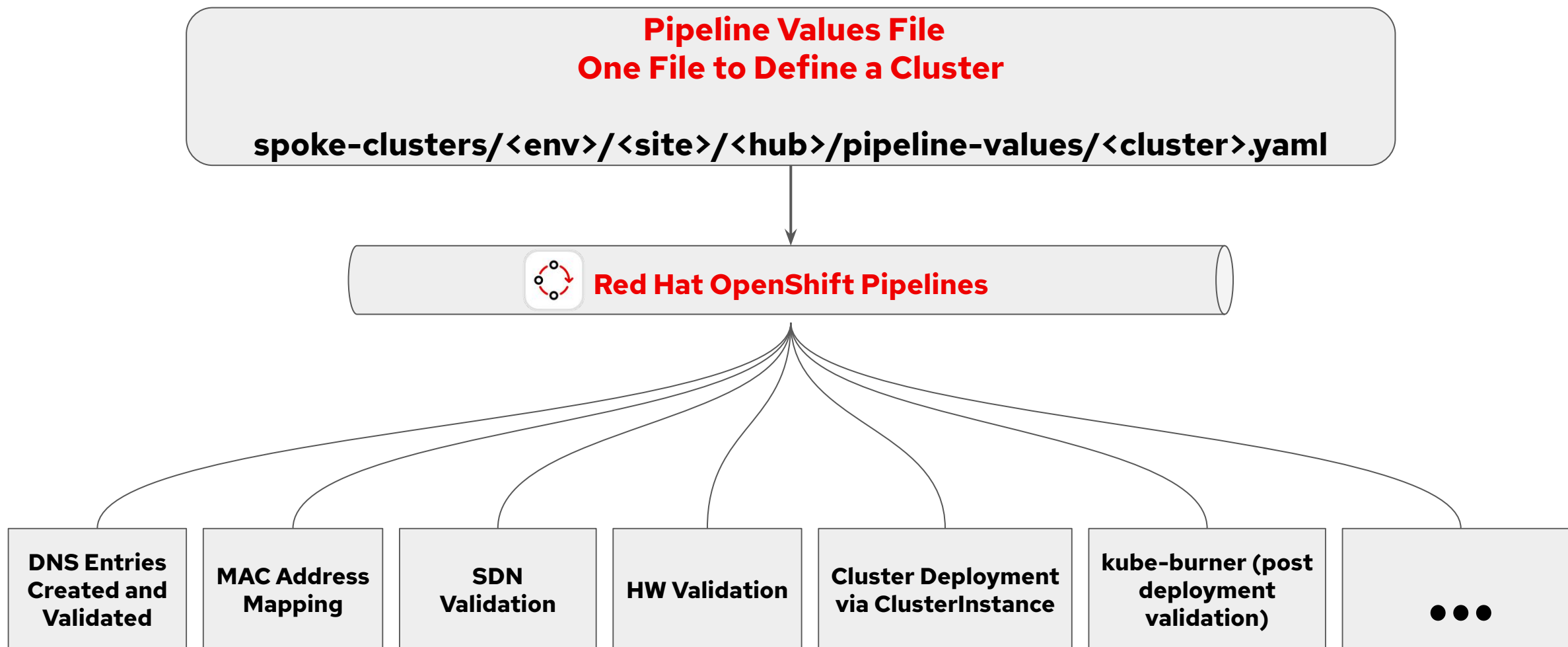
One missed field or type = broken cluster



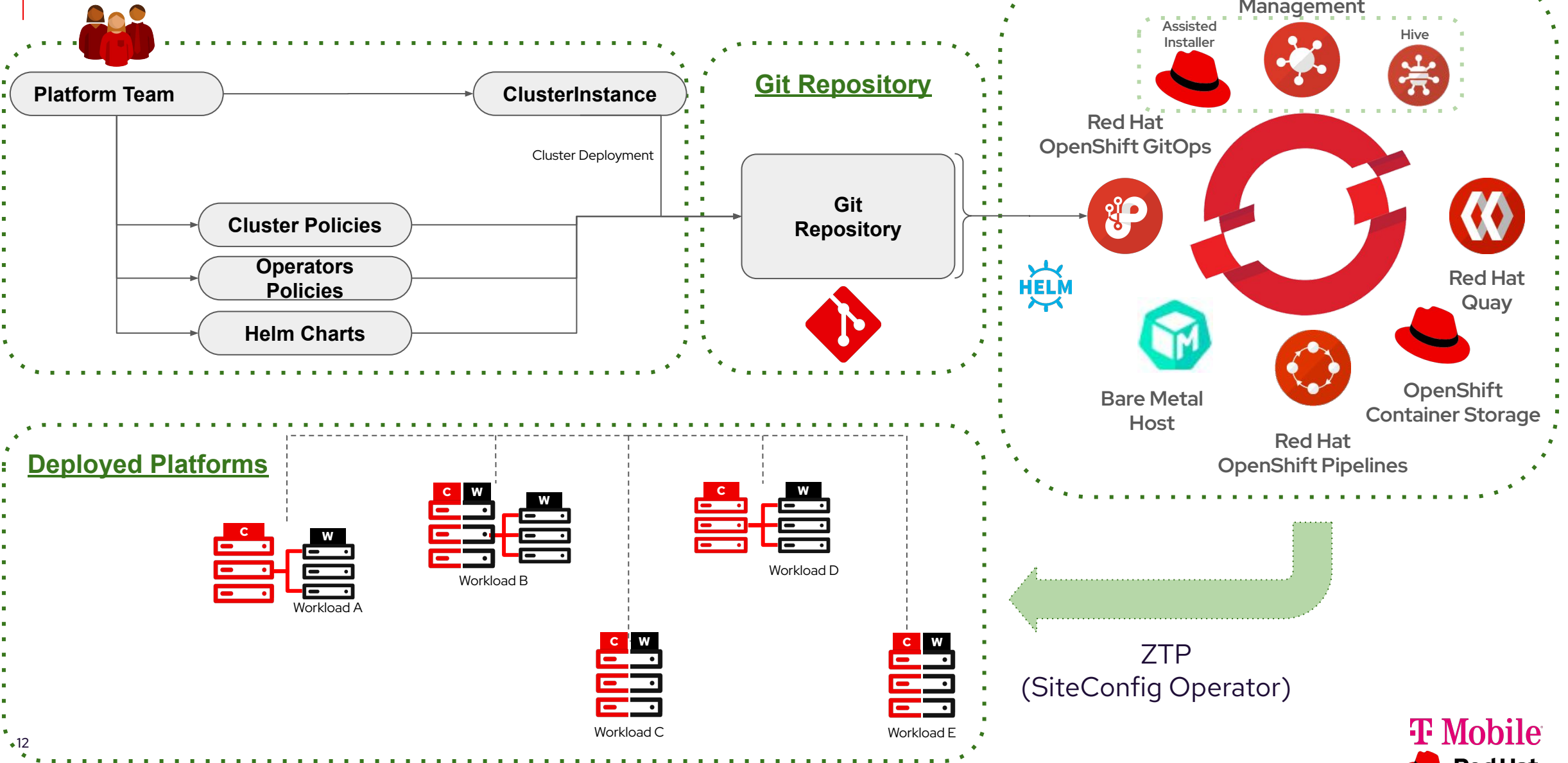
The Solution

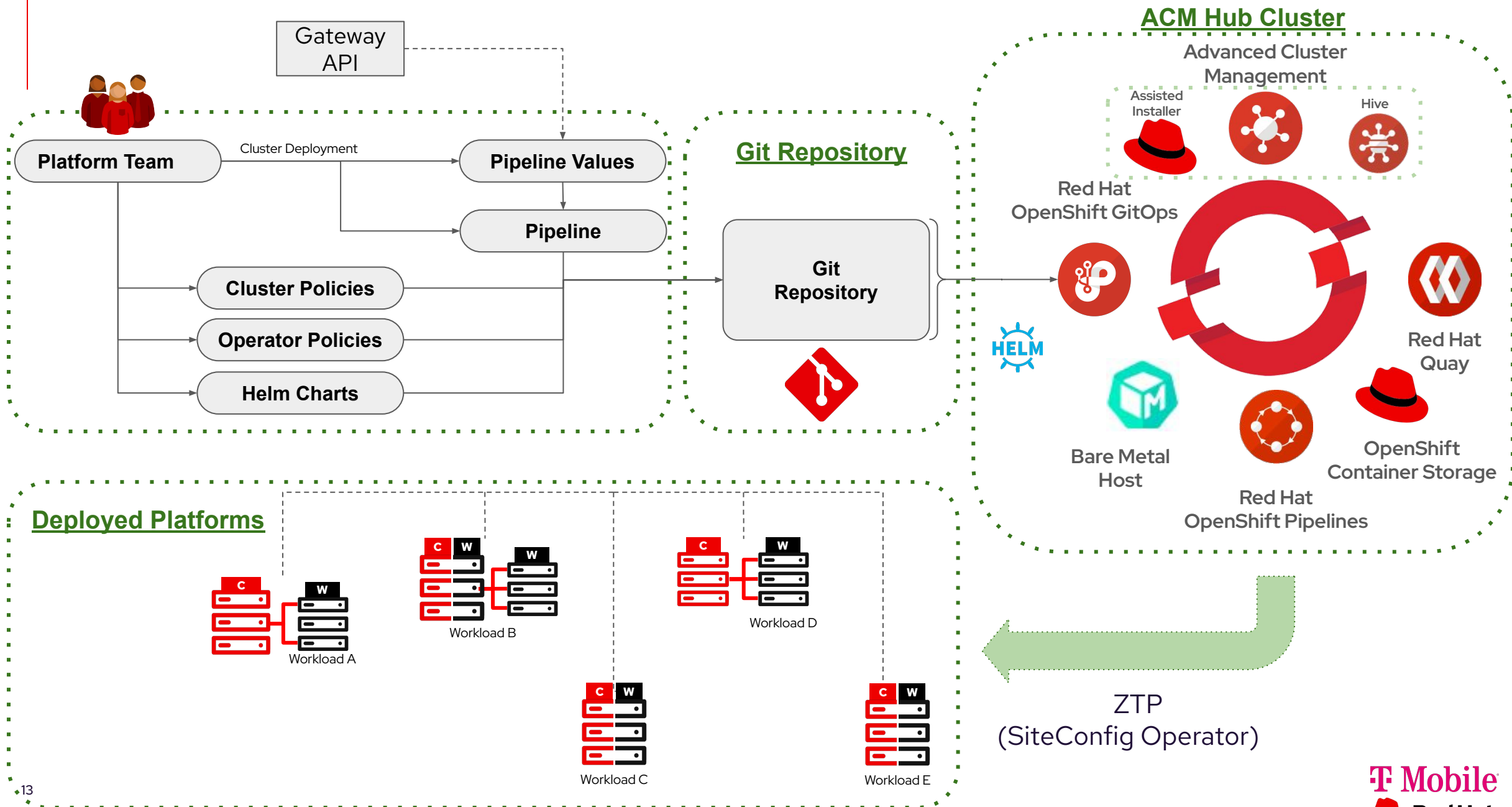


The Solution: Red Hat OpenShift Pipelines



GitOps + ACM ZTP (SiteConfig)





What is the Pipeline Values File?

A **single YAML file** that encodes **all business logic** for a cluster:

```
spoke-clusters/<env>/<site>/<acm-hub>/pipeline-values/<cluster-name>.yaml
```

```
clusterType: caas # Workload profile
clusterName: <cluster-name>
clusterImageSetNameRef: img4.20.0-x86-64...

nodeHostnames: # Topologies
masters: [node, node2, node3]
workers: [node4, node5, node6]
storage: [node7, node8, node9]

# Networking
clusterNetwork:
- cidr: <cidr>
```

```
clusterLabels:
  common: "true"

bond0Interfaces:
  masterNodeInterfacePorts:
    - eno12399np0
    - en2f1np1

clusterValues:
  CLForwarding: "enabled"
  bkupSched: "0 */12 * * *"

rbacGroupMappings:
```

This file encodes 100% of the cluster's business logic and serves as the definitive platform API

The pipeline reads it. Helm renders from it. GitOps deploys from it.

ACM Hub maintains it as a ConfigMap

Pipeline Values: Operator Version Management

Every operator installed on the spoke is **version-pinned** in the pipeline values.

```
clusterValues:
# OpenShift Data Foundation
odf-channel: "stable-4.20"
odf-operator-startingCSV: odf-operator.v4.20.9-rhodf
odf-approved-versions: |
- odf-operator.v4.20.9-rhodf

# OpenShift GitOps
openshift-gitops-channel: "latest"
openshift-gitops-startingCSV: openshift-gitops-operator.v1.19.1
openshift-gitops-approved-versions: |
- openshift-gitops-operator.v1.19.1

# MetalLB
metallb-channel: "stable"
metallb-startingCSV: metallb-operator.v4.20.0-202602091941
metallb-approved-versions: |
- metallb-operator.v4.20.0-202602091941

# ... 20+ operators, all version-controlled in one place
```

The Upgrade Workflow:

- Modify lines in the pipeline values file
- Git commit changes
- Open a Merge Request (MR)
- Run the pipeline automation
- Verify generated changes in MR

Upgrading an operator is a strictly controlled, git-auditable process.



Image Mirroring Pipeline

The Disconnected Deployment Challenge

T-Mobile's spoke clusters deploy in **air-gapped / disconnected environments**

No direct access to registry.redhat.io, quay.io, docker.io, etc.

Everything needed must be mirrored to the local Quay registry on the ACM Hub Cluster

This includes:

- OCP platform images (release payload, RHCOS)
- Operator catalogs and all operator bundle images
- Additional images (security agents, backup tools, custom workloads)

Without mirroring, workloads and platform containers will fail to start.

oc-mirror OpenShift Pipeline

Trigger: Manual or scheduled (before planned cluster deployments or operator upgrades)

oc-mirror-pipeline

- **fetch-ics** Fetch the ImageSetConfiguration from namespace
- **oc-mirror** `oc-mirror --v2 --config imagesetconfig.yaml docker://quay.local/mirror`
generates IDMS/ITMS ImageDigestMirrorSet resources
- **generate-manifests** Produce ITMS/IDMS as an output from oc-mirror
- **git-commit** Commit IDMS resources to Git repo, ACM policies dynamically
apply IDMS/ITSM to spoke clusters

Result: `clusterImageSetNameRef: img4.20.0-x86-64-appsub-mirror` resolves to local Quay for cluster deployments.

oc-mirror OpenShift Pipeline

Trigger: Application team can trigger pipeline via exposed API eventListener

oc-mirror-pipeline

- **clone-repo** Clone the image-mirror-pipeline config
- **oc-mirror** `oc mirror --v2 --config imagesetconfig.yaml docker://quay.local/mirror`
generates IDMS/ITMS ImageDigestMirrorSet resources and CatalogSource
- **generate-manifests** Produce mirror mapping ConfigMaps
- **git-commit** Commit IDMS resources to Git repo, ACM policies dynamically
apply IDMS/ITSM to spoke clusters

App teams requiring external operators/images can manage their own **imagesetconfiguration**. Using the managed pipeline to mirror into the hub registry without needing direct push access.



Cluster Build Pipeline

Stage 1: Clone Git Repos & Locate Pipeline Values

clone-repos

- `git clone spoke-clusters repo`
- `git clone cluster-build-automation repo`
- `find spoke-clusters/*/99-pipeline-values/<CLUSTER_NAME>.yaml`
- `emit: ENV, SITE, HUBNAME, cluster-exists (results)`

```
spoke-clusters/  
  dev/           ← ENV  
    us1/         ← SITE  
      us1-acm-hub1/ ← HUBNAME  
        99-pipeline-values/  
          us1-spoke1.yaml ← CLUSTER_NAME
```

The pipeline values file path encodes the cluster's location in the fleet

Stage 2: BMC Interrogation via Redfish

get-mac-addresses

- Read nodeHostnames from pipeline values file
- For each node: POST to BMC/iDRAC/iLO Redfish API
- Extract NIC MAC addresses (matching desiredNICs spec)
- Write MACs to workspace for generate-cluster-files

From the Pipeline Values:

desiredNICs:

- id: NIC.Integrated.1-1-1 # ← iDRAC NIC/PORT ID
name: eno12399np0 # ← maps to logical interface
- id: NIC.Slot.1-1-1
name: ens1f0np0

Why this matters:

- ClusterInstance requires exact MAC addresses for network config
- Manual MAC collection is error-prone and requires physical access
- Redfish enables zero-touch hardware interrogation

Automated hardware discovery replaces manual, physical data collection

Stage 3: Pre-Cluster Validation - Failing Fast

validate-hardware-task

- └─ Connect to node BMC
- └─ Check: BMC health
- └─ Check: BIOS config
- └─ Check: RAID status
- └─ Check: HW inventory
- └─ Fail fast

validate-sdn

- └─ Query SDN REST API
- └─ Validate contracts
- └─ Check subnets
- └─ Confirm hub↔spoke connectivity

validate-network

- └─ Gen custom RHCOS ISO with network config + scripts
- └─ Boot each node via Redfish
- └─ ⚠ **MANUAL APPROVAL GATE**
- └─ Run ping from each node
- └─ Verify routes DNS, NTP, Gateway, ACM Hub,
- └─ Collect results Unmount ISO
- └─ Unmount ISO
- └─ Shutdown

Fail fast. Identify issues before deployment

Stage 4: Generate Cluster Files (Helm Rendering)

generate-cluster-files

```
helm template helm-spoke/ \  
  --set-file clusterValues=<cluster>.yaml \  
  --set macAddresses=<output-from-get-mac-addresses>  
  
Renders:  
├─ clusterinstance.yaml    ← Full SiteConfig ClusterInstance CR  
├─ secrets.yaml            ← BMC secret, pull secret (Vault refs)  
├─ extra-manifests/        ← ConfigMaps with day-1 manifests  
└─ namespace.yaml
```

Write rendered files to: `spoke-clusters/<env>/<site>/<hub>/clusters/<cluster>/`

ClusterInstance CR Requirements

Contains every field ACM Assisted Installer requires: node BMC addresses, MAC→interface mappings, NMState network config, SSH public key, pull secret reference, `clusterImageSetNameRef`, labels.

No manual YAML editing. No copy-paste errors.
Pipeline values → Helm → ClusterInstance

Stage 5: GitOps Handoff -> MR -> Deploy

git-commit-and-mr

- `git add clusters/<cluster>/`
- `git commit -m "feat: add <cluster> cluster manifests"`
- `git push feature/<cluster>-<timestamp>`
- POST GitLab API: create merge request
 - source: feature/<cluster>
 - target: main

wait-for-merge

- Poll GitLab MR status every 30s
- OPEN → wait
- MERGED → proceed to deployment watch
- CLOSED → fail pipeline

deploy-cluster

- Watch ClusterInstance resource (ACM begins provisioning)
- Watch AgentClusterInstall for install progress
- Phases: Preparing → Installing → Finalizing
- Wait for: ClusterInstallCompleted = True

The merge IS the deployment trigger

GitOps detects the new ClusterInstance manifest in git and syncs it to the hub.
ACM Assisted Installer begins bare-metal provisioning

Stage 6: Post-Deployment Validation

Cluster Operator Validation:

`validate-cluster`

- Fetch kubeconfig for new spoke cluster
- Wait for all ClusterOperators: Available=True
- Wait for all MachineConfigPools: Updated=True
- Confirm: no degraded operators

`kube-burner (benchmarks)`

`run-kube-burner-tests`

- `kubelet-density` # Pod capacity
- `api-intensive` # API throughput
- `deployment-pvc-mv` # Storage I/O
- `network-policy` # CNI overhead
- `crd-scale` # CRD throughput

(1000+ objects per test, production-scale load)

Every cluster passes the same validation before entering the fleet.

- Results stored as pipeline artifacts
- Regressions are caught before workloads are deployed

Modifying Existing Clusters - post install

Pipeline Values File (us1-spoke1.yaml)

[Stage 1]-	clone-repos	Locate pipeline values and determine if cluster already exists
[Stage 2]-	get-mac-addresses	Redfish BMC interrogation for new/replaced nodes MACs
[Stage 3]-	verify dns for all hosts - modify or create new dns records	
[Stage 4]-	retrieve-ssh-key	retrieve the live ssh key from vault
	generate-cluster-files	Helm → ClusterInstance CR and per cluster configmap containing unique cluster configuration
[Stage 5]-	git-commit-and-mr	Feature branch + GitLab MR
	wait-for-merge	Human review + merge

Pipeline Values -> Day2 configuration updated via pipeline and ACM policies

Bulk Modification

Using the bulk modification pipeline, we are able to update the day2 configuration of all clusters under single hub cluster by a single pipeline run.

Rollout strategies - The bulk modification pipeline supports rolling out in batches and using an exclusion list to deny the modification of any cluster with a specific name

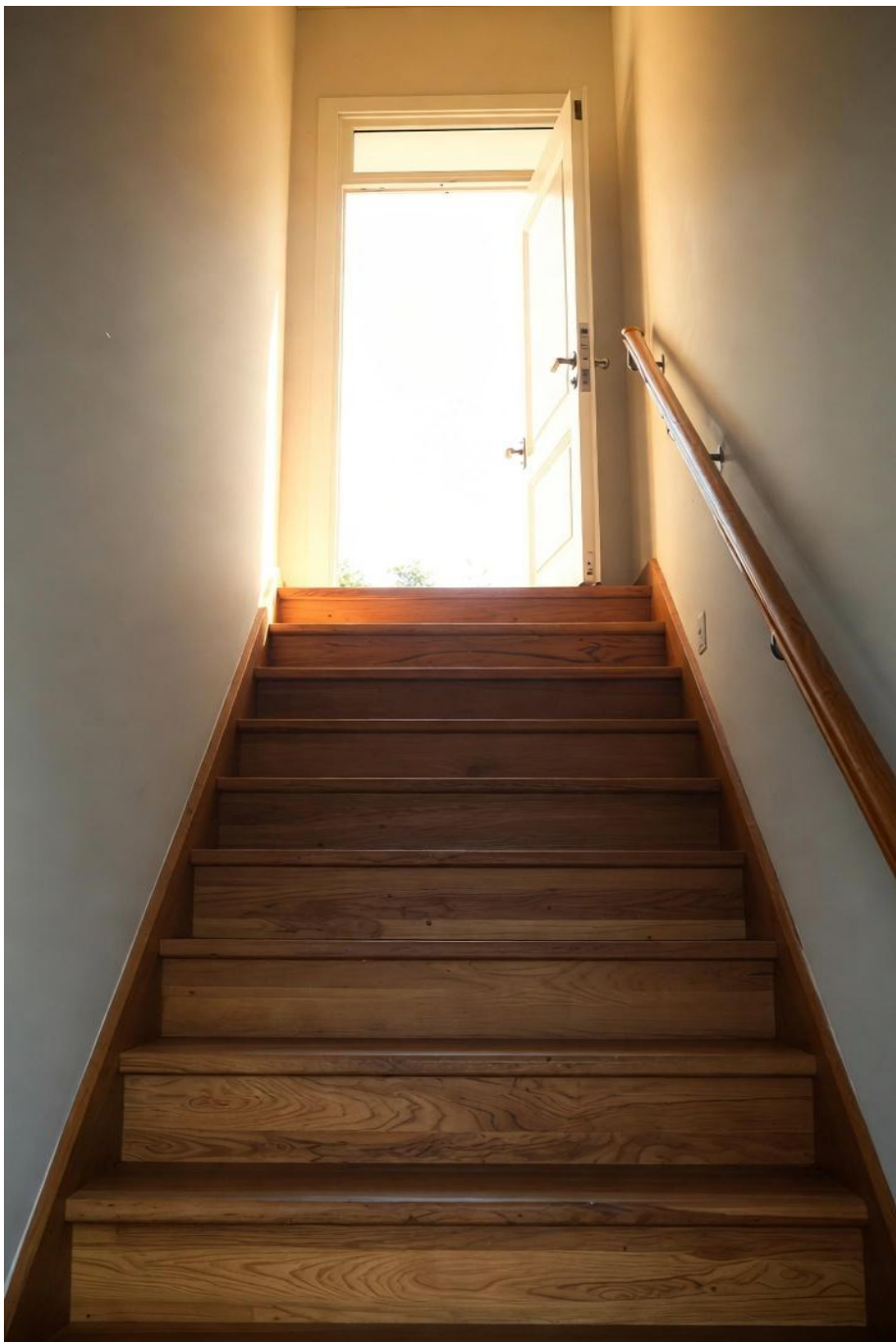
```
pipeline-params:
- name: EXCLUDE
  value: 'us-spoke1'
- name: MAX_CONCURRENT
  value: '5'
- name: DRY_RUN <- Output the clusters that would be modified
  value: 'false'
- name: RUN_SDN_PRECHECKS
  value: 'false'
- name: RUN_HARDWARE_VALIDATION
  value: 'false'
- name: RUN_NETWORK_CONNECTIVITY_TEST
  value: 'false'
- name: RUN_KUBE_BURNER_TESTS
  value: 'false'
```

Node Replacement

Use the cluster build pipeline to re-deploy a node.

Pipeline Values File (us1-spoke1.yaml)

[Stage 1]-	clone-repos	Check if cluster exists
[Stage 2]-	get-mac-addresses	BMC query new node for MACs
[Stage 3]-	verify-dns	Check new node DNS
[Stage 4]-	retrieve-ssh-key	Fetch Vault key
	generate-cluster-files	Helm → ClusterInstance CR
[Stage 5]-	node-replacement	Remove OSD (if storage), etcd (if master), & suppress BMH on ClusterInstance, delete node
[Stage 6]-	git-commit-and-mr	GitLab MR
	wait-for-merge	Review & merge
[Stage 7]-	unsuppress-bmc	Unsuppress BMH & provision new host



Next Steps



Next Steps: API driven Cluster-as-a-Service

Today's workflow: The pipeline values file defines a stable API contract

Human → Edit Pipeline Values File → Git Commit → GitLab MR → Pipeline → Cluster

Future workflow:

API Consumer → POST Gateway API → Validate → Pipeline → Cluster

```
// POST /api/v1/clusters
{
  "clusterName": "us1-spoke3",
  "clusterType": "caas",
  "site": "us1",
  "environment": "dev",
  "nodeHostnames": {
    "masters": ["node1", "node2", "node3"]
  },
  "clusterImageSet": "4.20",
  "operatorProfile": "standard-caas-v1",
  "networkProfile": "ipv6-sdn-standard",
  "entraGroups": { "admins": "...", "readonly": "..." }
}
```

Key Takeaways

1. GitOps as the Foundation

Git servers as the single source of truth for all cluster configuration and lifecycle management

2. Pipeline Values = Infrastructure API

A single YAML file encodes 100% of the business logic for a cluster deployment. Git is the API.

3. OpenShift Pipelines enable Autonomous GitOps

End-to-end pipelines eliminate manual processes and reduce operational risk

4. Built for Telecom Environments

Designed for air-gapped, distributed infrastructure with strict validation requirements

5. Future: API driven Cluster-as-a-Service

Moving toward fully API driven infrastructure consumption.

Try it yourself

We have created a repo on GitHub that contains the demo source code for this automation

<https://github.com/aprincipRedhat/rh-summit-gitops-tmobile/>





Thank you



linkedin.com/company/red-hat



facebook.com/redhat



instagram.com/red_hat



x.com/RedHatSummit